FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

RAPPORT DE PROJET DEVOPS

MASTER D'EXCELLENCE EN INTELLIGENCE ARTIFICIELLE

Mise en place d'une infrastructure DevOps automatisée

CI/CD, Kubernetes (K3s) et déploiement Cloud
d'une plateforme de prédiction AQI

Présenté par :

ZOUHAIRI Youness
MOUSSAOUI Abdelkarim

Sous la direction de :

Prof. BENABBOU Fouzia

Année Universitaire 2025 – 2026

Table des matières

Résumé **6**

Abstract **7**

1 Introduction générale **8**
1.1 Contexte du projet 8
1.2 Problématique 8
1.3 Objectifs du projet 9
1.4 Structure du rapport 9

2 Étude de l'existant et choix technologiques **10**
2.1 Panorama des outils DevOps 10
2.1.1 Plateforme cloud 10
2.1.2 Infrastructure as Code 10
2.1.3 Conteneurisation et orchestration 11
2.1.4 CI/CD 11
2.1.5 Monitoring 12
2.2 Architecture technique retenue 12

3 Architecture du système **13**
3.1 Architecture globale du système 13
3.2 Architecture de l'infrastructure cloud 14
3.3 Architecture Kubernetes 15
3.4 Architecture applicative 17
3.5 Pipeline CI/CD 19
3.6 Déploiement de l'application sur Kubernetes 21

4 Mise en place de l'infrastructure et déploiement **22**
4.1 Préparation de l'environnement de développement 22
4.1.1 Installation de l'AWS CLI 22
4.1.2 Configuration des identifiants AWS 23
4.1.3 Création et téléchargement de la clé SSH 24
4.1.4 Installation de Terraform 25
4.1.5 Installation d'Ansible sur WSL 25
4.1.6 Structure du projet 26

4.2 Provisionnement de l'infrastructure avec Terraform 26
4.2.1 Chaîne d'automatisation entre AWS CLI, Terraform, SSH et Ansible . . 26
4.2.2 Configuration du provider AWS 28
4.2.3 Configuration des règles de sécurité 28
4.2.4 Création des instances EC2 29
4.3 Configuration automatique avec Ansible 30
4.3.1 Inventaire Ansible 30
4.3.2 Connexion SSH utilisée par Ansible 30
4.3.3 Exécution du playbook 30
4.3.4 Création d'un fichier swap 31
4.4 Déploiement du cluster Kubernetes 32
4.4.1 Vérification du cluster 32
4.5 Déploiement de l'application 32
4.5.1 Deployment du backend 33
4.5.2 Service backend 33
4.5.3 Deployment du frontend 33
4.5.4 Service frontend 33

5 Application conteneurisée et pipeline CI/CD 34
5.1 Dépôt GitHub du projet 34
5.2 Architecture de l'application 35
5.3 Conteneurisation avec Docker 35
5.3.1 Conteneurisation du backend 35
5.3.2 Conteneurisation du frontend 36
5.4 Publication des images Docker 36
5.5 Pipeline CI/CD avec GitHub Actions 37
5.6 Authentification de GitHub Actions au cluster Kubernetes 37
5.7 Pipeline conditionnel 38
5.8 Résultat du pipeline GitHub Actions 39
5.9 Déploiement automatique sur Kubernetes 39
5.10 Limites de l'infrastructure 40

6 Conclusion générale et perspectives 41
6.1 Difficultés rencontrées 41
6.1.1 Contraintes de ressources des instances cloud 41
6.1.2 Accès à l'API Kubernetes 42
6.1.3 Problèmes de communication réseau entre les nœuds 42
6.1.4 Accès externe à l'application 42
6.1.5 Crash des conteneurs Nginx 43
6.1.6 Résolution DNS dans Kubernetes 43

6.1.7 Problèmes de routage des requêtes 43
6.1.8 Erreurs de type 502 Bad Gateway 43
6.1.9 Synchronisation des services Kubernetes 43
6.1.10 Optimisation du pipeline CI/CD 44
6.2 Apports du projet 44
6.3 Perspectives d'amélioration 44
6.3.1 Ajout d'un système de monitoring 44
6.3.2 Mise en place d'un Ingress Controller 45
6.3.3 Extension du modèle de prédiction 45
6.4 Conclusion 45

Table des figures

3.1 Architecture globale de la plateforme DevOps 14
3.2 Architecture de l'infrastructure AWS 15
3.3 Architecture interne du cluster Kubernetes 16
3.4 Architecture applicative de la plateforme AQI Predictor 18
3.5 Pipeline CI/CD du projet 20

4.1 Vérification de l'installation de AWS CLI 23
4.2 Configuration des identifiants AWS avec AWS CLI 23
4.3 Création ou sélection de la paire de clés SSH dans AWS 24
4.4 Vérification de l'installation de Terraform 25
4.5 Installation d'Ansible dans WSL 26
4.6 Chaîne d'automatisation entre AWS CLI, Terraform, SSH et Ansible 27
4.7 Configuration du Security Group AWS 28
4.8 Instances EC2 utilisées pour le cluster Kubernetes 29
4.9 Exécution du playbook Ansible 31
4.10 Vérification des nœuds du cluster Kubernetes 32
4.11 Pods de l'application AQI dans Kubernetes 33

5.1 Images Docker publiées sur Docker Hub 37
5.2 Stockage sécurisé du kubeconfig dans GitHub Secrets 38
5.3 Résultat de l'exécution du pipeline CI/CD 39
5.4 Redéploiement automatique des pods Kubernetes 40

Liste des tableaux

2.1 Spécifications des machines virtuelles AWS 12

Résumé

Mots-clés : DevOps, CI/CD, Kubernetes, K3s, Docker, Terraform, Ansible, GitHub Actions, FastAPI, React, AQI

Ce projet présente la conception et la mise en place d'une infrastructure DevOps complète permettant le déploiement automatisé d'une application web de prédiction de l'indice de qualité de l'air (AQI).

L'objectif principal du projet est de démontrer l'utilisation de pratiques DevOps modernes afin d'automatiser l'ensemble du cycle de vie d'une application, depuis le développement du code jusqu'à son déploiement dans un environnement cloud.

L'application développée repose sur une architecture composée de deux parties principales : un backend développé avec le framework FastAPI permettant de calculer et d'estimer l'indice AQI, et un frontend développé avec React et Vite offrant une interface utilisateur interactive.

L'ensemble du système est conteneurisé à l'aide de Docker afin d'assurer la portabilité et la reproductibilité de l'environnement d'exécution. L'application est ensuite déployée sur un cluster Kubernetes léger basé sur K3s, composé d'un nœud maître et d'un nœud worker hébergés sur des instances AWS EC2.

L'infrastructure cloud est provisionnée automatiquement à l'aide de Terraform, tandis que la configuration des serveurs et l'installation du cluster Kubernetes sont automatisées avec Ansible.

Un pipeline CI/CD implémenté avec GitHub Actions automatise les différentes étapes du cycle de vie applicatif : exécution des tests, construction des images Docker, publication des images sur DockerHub et mise à jour des déploiements Kubernetes.

L'application est accessible via un service Kubernetes de type NodePort exposant le frontend, lequel agit comme proxy vers le backend grâce à une configuration Nginx.

Ce projet illustre ainsi l'intégration de plusieurs technologies DevOps modernes afin de construire une infrastructure automatisée, reproductible et adaptée au déploiement d'applications cloud-native.

Abstract

This project presents the design and implementation of a complete DevOps infrastructure for the automated deployment of an Air Quality Index (AQI) prediction platform.

The main objective of this work is to demonstrate how modern DevOps practices can be applied to automate the entire lifecycle of an application, from development to deployment in a cloud environment.

The developed platform consists of two main components : a FastAPI backend responsible for computing and estimating AQI values, and a React-based frontend providing an interactive user interface.

Both components are containerized using Docker in order to ensure portability and reproducibility across environments. The application is deployed on a lightweight Kubernetes cluster built with K3s, composed of a control plane node and a worker node running on AWS EC2 instances.

Infrastructure provisioning is automated using Terraform, while server configuration and Kubernetes installation are handled using Ansible.

A CI/CD pipeline implemented with GitHub Actions automates application testing, Docker image building, publishing images to DockerHub, and updating Kubernetes deployments.

The application is exposed to users through a Kubernetes NodePort service that routes HTTP requests to the frontend pod. The frontend uses Nginx to proxy API requests to the backend service running inside the cluster.

This project demonstrates how modern DevOps tools such as Docker, Kubernetes, Terraform, Ansible, and GitHub Actions can be integrated to build a fully automated and reproducible cloud-native deployment infrastructure.

Chapitre 1

Introduction générale

1.1 Contexte du projet

L'évolution rapide des technologies cloud et des architectures distribuées a profondément transformé les méthodes de développement et de déploiement des applications modernes. Dans ce contexte, les pratiques DevOps jouent un rôle central en permettant de rapprocher les équipes de développement et d'exploitation afin d'améliorer la qualité, la fiabilité et la rapidité de livraison des logiciels.

Les applications modernes sont de plus en plus déployées sous forme de conteneurs afin de garantir la portabilité des environnements et de simplifier la gestion des dépendances. Docker s'est imposé comme la solution de référence pour la conteneurisation des applications, tandis que Kubernetes est devenu la plateforme dominante pour l'orchestration de conteneurs à grande échelle.

Par ailleurs, l'automatisation de l'infrastructure à l'aide d'outils d'Infrastructure as Code (IaC) tels que Terraform et l'automatisation de la configuration avec des outils comme Ansible permettent de déployer des infrastructures reproductibles et facilement maintenables.

Ce projet s'inscrit dans cette démarche en proposant la mise en place d'une infrastructure DevOps automatisée permettant de déployer une application de prédiction de la qualité de l'air dans un environnement cloud.

1.2 Problématique

Le déploiement manuel d'applications dans un environnement cloud présente plusieurs limites : manque de reproductibilité, risques d'erreurs humaines, difficulté de mise à l'échelle et absence d'automatisation dans le cycle de déploiement.

Dans un contexte où les applications doivent évoluer rapidement et être déployées fréquemment, il devient nécessaire de mettre en place des pipelines automatisés capables d'assurer l'intégration et le déploiement continus des applications.

La problématique principale de ce projet peut donc être formulée de la manière suivante :

Comment concevoir et déployer une infrastructure DevOps automatisée permettant de gérer le cycle complet de développement, de conteneurisation et de déploiement d'une application sur un cluster Kubernetes hébergé dans le cloud ?

1.3 Objectifs du projet

L'objectif principal de ce projet est de concevoir et de mettre en place une infrastructure DevOps complète permettant le déploiement automatisé d'une application conteneurisée.

Les objectifs spécifiques du projet sont les suivants :

— Développer une application web de prédiction de l'indice de qualité de l'air (AQI).

— Conteneuriser l'application à l'aide de Docker afin de garantir la portabilité des environnements.

— Provisionner une infrastructure cloud sur AWS à l'aide de Terraform.

— Automatiser la configuration des serveurs et l'installation du cluster Kubernetes avec Ansible.

— Déployer l'application sur un cluster Kubernetes léger basé sur K3s.

— Mettre en place un pipeline CI/CD automatisé avec GitHub Actions.

— Publier les images Docker sur DockerHub.

— Automatiser la mise à jour des déploiements Kubernetes lors des nouvelles versions de l'application.

1.4 Structure du rapport

Le présent rapport est organisé en plusieurs chapitres décrivant les différentes étapes de la réalisation du projet.

Le chapitre suivant présente l'architecture globale du système et les technologies utilisées dans l'infrastructure DevOps.

Le chapitre consacré à la mise en place de l'infrastructure décrit l'utilisation de Terraform et d'Ansible pour provisionner et configurer les ressources cloud.

Un chapitre est ensuite dédié à la conteneurisation de l'application et à son déploiement sur Kubernetes.

Le pipeline CI/CD mis en place avec GitHub Actions est également détaillé afin d'expliquer le processus automatisé de test, de construction et de déploiement des images Docker.

Enfin, le rapport présente les principales difficultés rencontrées durant le projet ainsi que les solutions apportées, avant de conclure sur les résultats obtenus et les perspectives d'amélioration.

Chapitre 2

Étude de l'existant et choix technologiques

2.1 Panorama des outils DevOps

L'écosystème DevOps regroupe un grand nombre d'outils permettant d'automatiser les différentes étapes du cycle de vie d'une application : provisionnement de l'infrastructure, configuration des serveurs, conteneurisation, orchestration et déploiement continu.

Dans le cadre de ce projet, plusieurs solutions ont été étudiées afin de sélectionner les technologies les plus adaptées aux contraintes du projet, notamment les ressources limitées de l'infrastructure cloud utilisée.

2.1.1 Plateforme cloud

Plusieurs plateformes cloud proposent des environnements permettant d'héberger des infrastructures DevOps et des clusters Kubernetes. Parmi les solutions les plus répandues figurent Amazon Web Services (AWS), Google Cloud Platform (GCP), Microsoft Azure et Oracle Cloud Infrastructure.

Dans ce projet, la plateforme Amazon Web Services (AWS) a été retenue pour le déploiement de l'infrastructure. AWS offre une grande flexibilité dans la gestion des machines virtuelles grâce au service EC2, ainsi qu'une large documentation et un écosystème mature pour le déploiement d'applications cloud-native.

Les machines virtuelles utilisées sont des instances EC2 de type *t3.micro*, incluses dans l'offre gratuite AWS Free Tier. Bien que ces instances offrent des ressources limitées (1 vCPU et 1 GB de RAM), elles permettent de mettre en place un environnement DevOps fonctionnel dans un cadre académique.

2.1.2 Infrastructure as Code

L'Infrastructure as Code (IaC) consiste à gérer et provisionner l'infrastructure informatique à l'aide de fichiers de configuration versionnés, plutôt que par des opérations manuelles.

Dans ce projet, Terraform a été utilisé pour automatiser la création des ressources cloud sur AWS. Terraform permet de décrire l'infrastructure sous forme déclarative à l'aide du langage HCL (HashiCorp Configuration Language). Cette approche garantit la reproductibilité et la traçabilité de l'infrastructure.

Afin de compléter Terraform, Ansible a été utilisé pour l'automatisation de la configuration des serveurs. Tandis que Terraform se charge de créer les machines virtuelles, Ansible permet d'installer automatiquement les dépendances nécessaires, notamment Docker et Kubernetes K3s.

2.1.3 Conteneurisation et orchestration

La conteneurisation est devenue un standard dans le déploiement des applications modernes. Docker permet d'encapsuler une application ainsi que toutes ses dépendances dans une image portable pouvant être exécutée sur n'importe quel environnement compatible.

Cependant, lorsque plusieurs conteneurs doivent être exécutés et gérés à grande échelle, un système d'orchestration devient nécessaire. Kubernetes est aujourd'hui la solution de référence pour l'orchestration de conteneurs.

Dans ce projet, la distribution Kubernetes légère *K3s* a été choisie. K3s est une version simplifiée de Kubernetes conçue pour les environnements à ressources limitées. Contrairement à une installation Kubernetes classique, K3s consomme beaucoup moins de mémoire et de CPU, ce qui le rend adapté aux instances EC2 de type t3.micro utilisées dans ce projet.

Le cluster Kubernetes est constitué de deux nœuds :

— Un nœud maître (control plane) responsable de la gestion du cluster

— Un nœud worker hébergeant les pods applicatifs

2.1.4 CI/CD

L'intégration et le déploiement continus (CI/CD) permettent d'automatiser les différentes étapes de développement, de test et de déploiement d'une application.

Plusieurs solutions existent pour mettre en place un pipeline CI/CD, notamment Jenkins, GitLab CI/CD et GitHub Actions.

Dans ce projet, GitHub Actions a été retenu pour sa simplicité d'utilisation et son intégration native avec les dépôts GitHub. Les pipelines sont définis directement dans le dépôt sous forme de fichiers YAML, ce qui permet de versionner l'ensemble du processus d'automatisation.

Le pipeline CI/CD implémenté dans ce projet réalise les étapes suivantes :

— Exécution des tests backend avec Pytest

— Exécution des tests frontend avec Vitest

— Construction des images Docker du frontend et du backend

— Publication des images Docker sur DockerHub

— Mise à jour des déploiements Kubernetes via *kubectl rollout restart*

2.1.5 Monitoring

Dans les environnements Kubernetes de production, il est courant d'utiliser des outils de monitoring tels que Prometheus et Grafana afin de superviser les performances des applications et des ressources système.

Cependant, ces outils nécessitent une quantité importante de ressources mémoire et CPU. Les instances EC2 utilisées dans ce projet étant de type t3.micro, elles disposent de ressources limitées (1 GB de RAM). L'installation d'une stack de monitoring complète aurait donc entraîné une surcharge du cluster Kubernetes.

Pour cette raison, aucune solution de monitoring avancée n'a été déployée dans ce projet. Le choix de la distribution Kubernetes légère K3s permet néanmoins de réduire la consommation des ressources système et de maintenir un cluster fonctionnel malgré les contraintes matérielles.

2.2 Architecture technique retenue

L'architecture finale du système repose sur une infrastructure cloud composée de deux machines virtuelles EC2 hébergées sur AWS et formant un cluster Kubernetes K3s.

Machine	Instance	Rôle	Description
k3s-master	t3.micro	Control Plane	Gestion du cluster Kubernetes
k3s-worker	t3.micro	Worker Node	Hébergement des pods applicatifs

Table 2.1 – Spécifications des machines virtuelles AWS

Le cluster Kubernetes héberge deux composants principaux de l'application :

- Un frontend basé sur React et servi par Nginx
- Un backend développé avec FastAPI contenant le modèle de prédiction AQI

Le frontend est exposé au public via un service Kubernetes de type NodePort, tandis que le backend est accessible uniquement à l'intérieur du cluster via un service ClusterIP.

Le pipeline CI/CD déployé avec GitHub Actions permet d'automatiser le déploiement des nouvelles versions de l'application. À chaque mise à jour du code source dans le dépôt GitHub, les images Docker sont reconstruites, publiées sur DockerHub puis déployées automatiquement sur le cluster Kubernetes.

Chapitre 3

Architecture du système

Ce chapitre présente l'architecture globale de la plateforme DevOps mise en place pour le déploiement de l'application AQI Predictor. L'objectif est de décrire les différents composants du système ainsi que leurs interactions, depuis l'infrastructure cloud jusqu'au pipeline d'intégration et de déploiement continu.

L'architecture repose sur plusieurs technologies modernes, notamment Docker pour la conteneurisation, Kubernetes pour l'orchestration des conteneurs, Terraform pour la gestion de l'infrastructure cloud, Ansible pour l'automatisation de la configuration des serveurs, ainsi que GitHub Actions pour la mise en place du pipeline CI/CD.

3.1 Architecture globale du système

La figure suivante présente l'architecture globale du système, incluant l'infrastructure cloud, le cluster Kubernetes, l'application conteneurisée ainsi que le pipeline CI/CD.

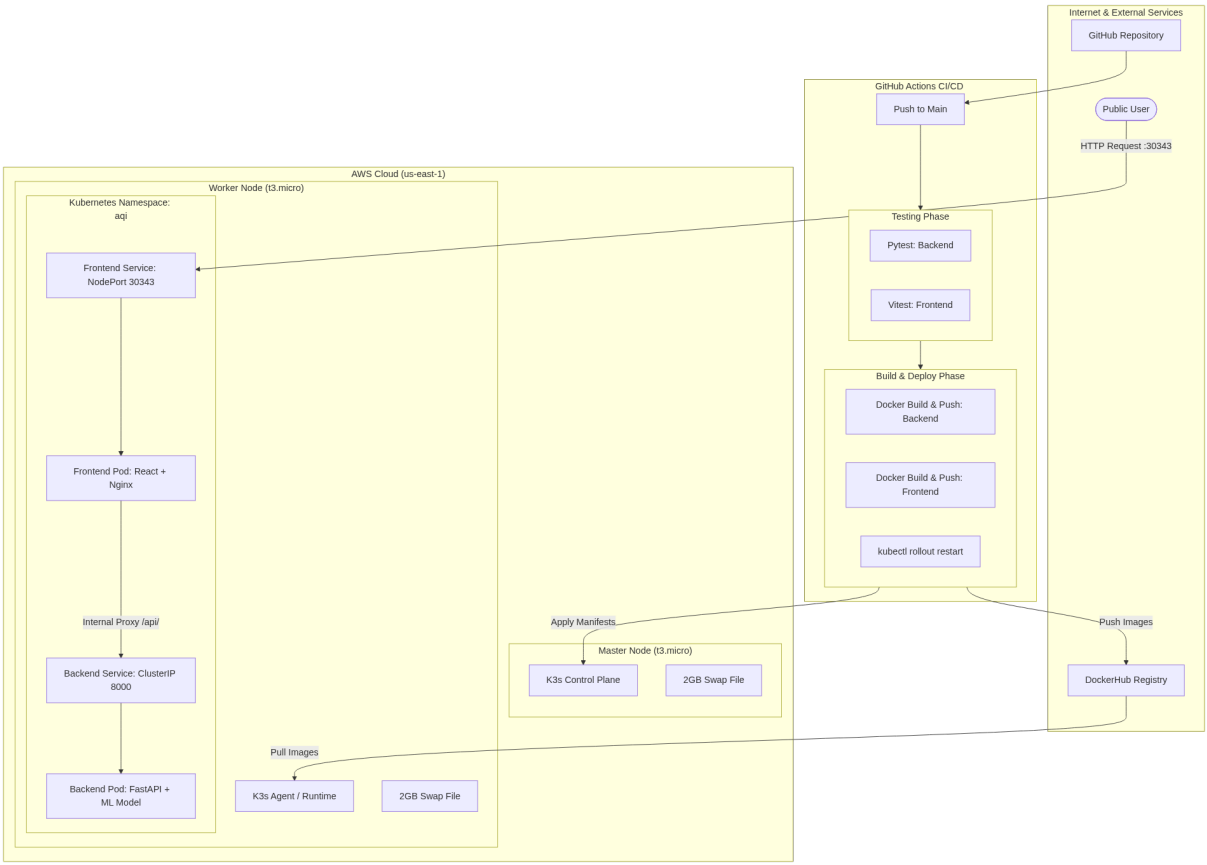

FIGURE 3.1 – Architecture globale de la plateforme DevOps

Dans cette architecture, l'application est déployée sur un cluster Kubernetes basé sur K3s, hébergé sur des instances EC2 dans le cloud AWS. L'application est composée d'un frontend React servant d'interface utilisateur et d'un backend FastAPI responsable du calcul de l'indice de qualité de l'air.

Les différentes images Docker de l'application sont construites et publiées automatiquement via un pipeline CI/CD implémenté avec GitHub Actions. Ces images sont ensuite récupérées par le cluster Kubernetes afin de mettre à jour les déploiements en cours.

3.2 Architecture de l'infrastructure cloud

L'infrastructure cloud du projet est hébergée sur Amazon Web Services (AWS). La création des ressources cloud est automatisée grâce à Terraform, permettant une gestion déclarative et reproductible de l'infrastructure.

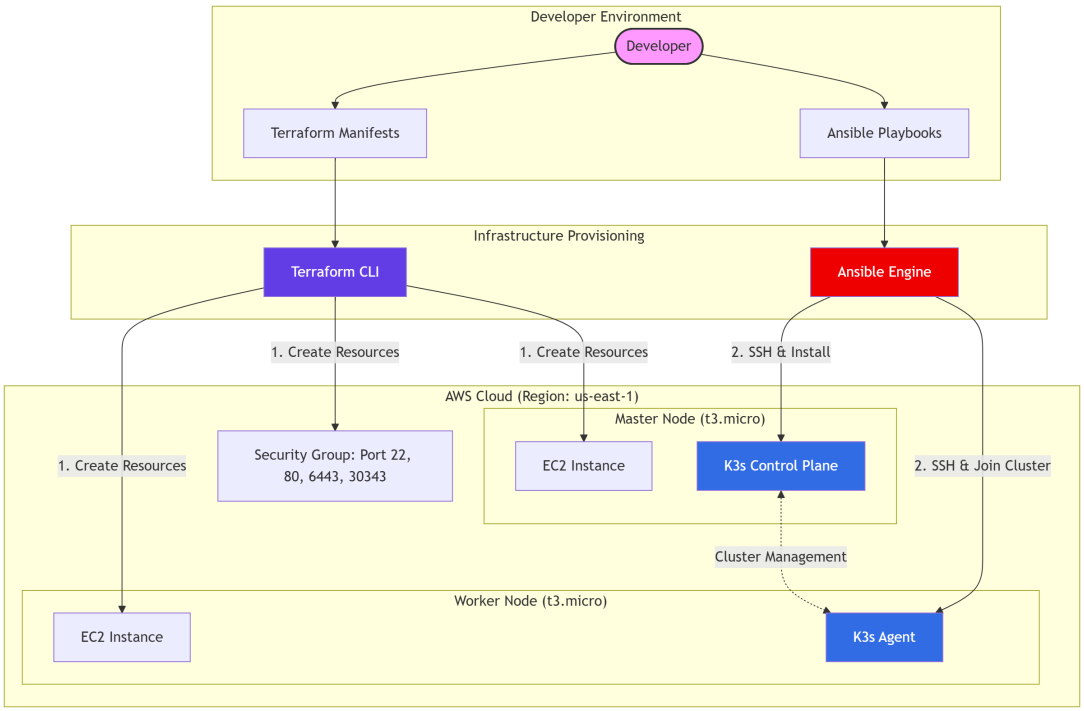

FIGURE 3.2 – Architecture de l'infrastructure AWS

Deux instances EC2 sont utilisées dans l'infrastructure : une instance jouant le rôle de nœud maître (control plane) et une instance servant de nœud worker. Le nœud maître exécute les composants principaux du cluster K3s, tandis que le nœud worker héberge les pods applicatifs.

Ansible est utilisé pour automatiser la configuration des serveurs et l'installation du cluster Kubernetes K3s.

3.3 Architecture Kubernetes

L'application est déployée dans un cluster Kubernetes léger basé sur K3s. Kubernetes permet de gérer le cycle de vie des conteneurs, d'assurer la scalabilité des services et de faciliter le déploiement continu des applications.

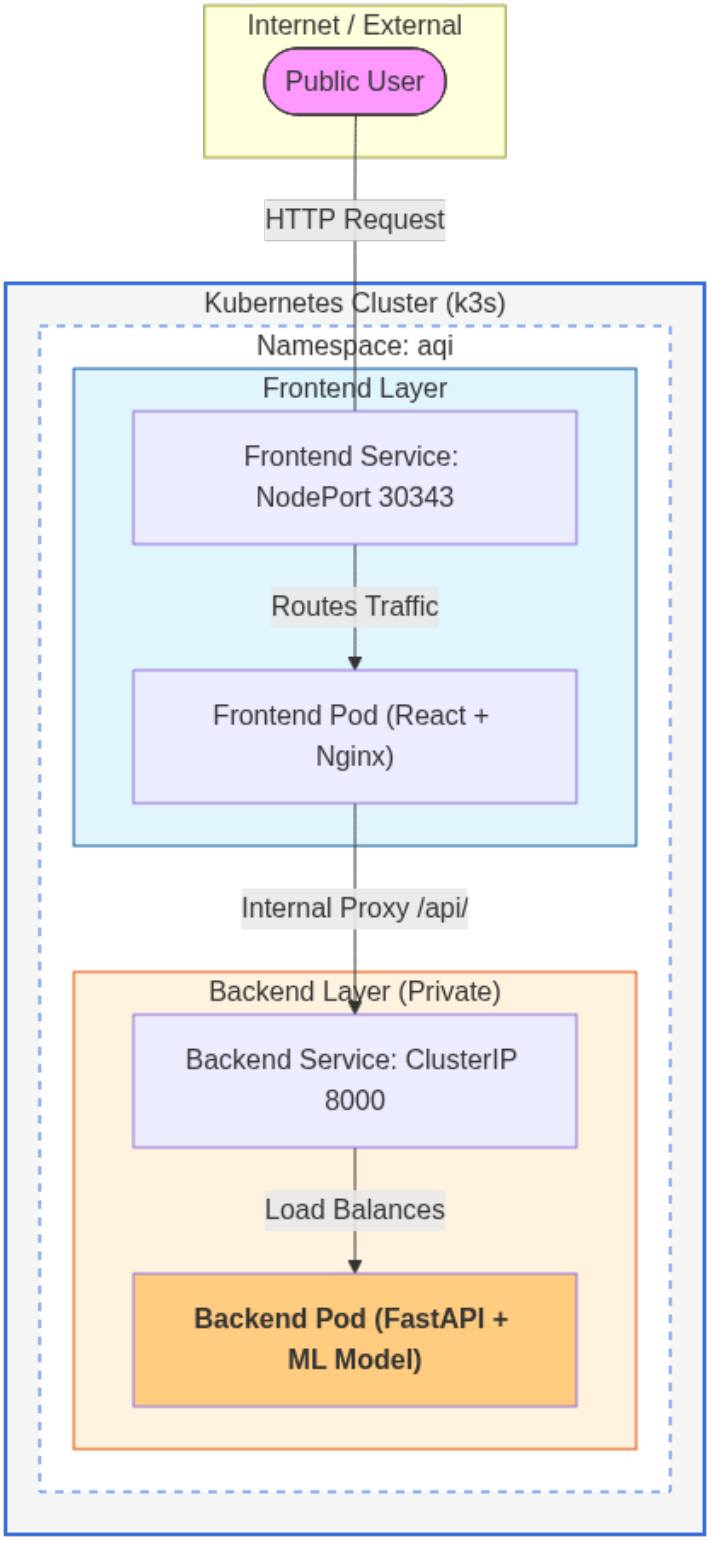

FIGURE 3.3 – Architecture interne du cluster Kubernetes

Le frontend est exposé au public via un service Kubernetes de type NodePort. Ce service permet aux utilisateurs d'accéder à l'application web via le port 30343.

Le frontend agit comme un proxy interne et redirige les requêtes API vers le backend FastAPI via un service Kubernetes de type ClusterIP.

3.4 Architecture applicative

L'application AQI Predictor est structurée selon une architecture client-serveur composée d'une interface utilisateur web et d'une API backend.

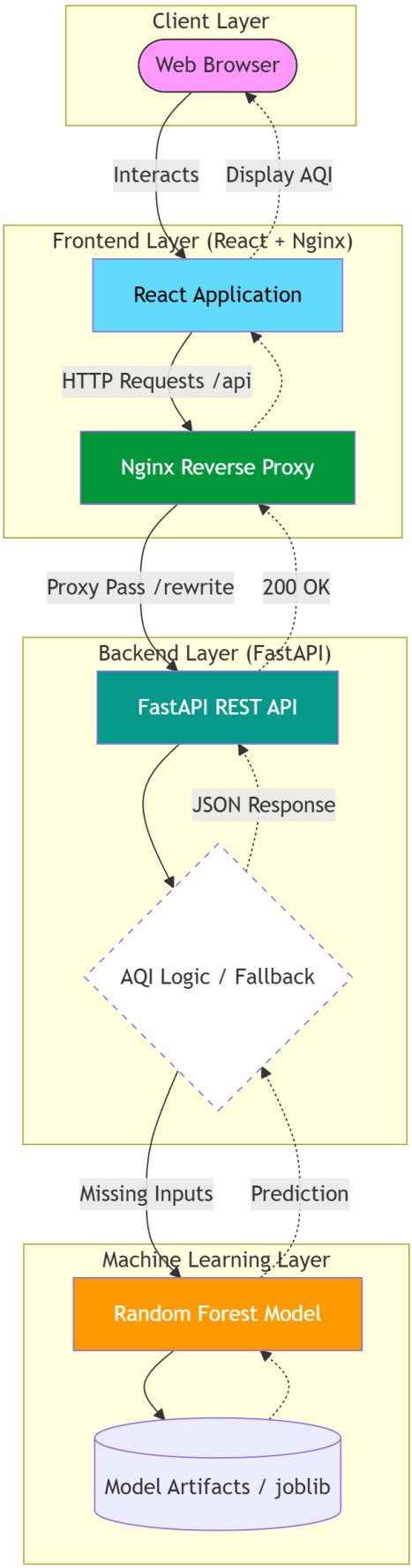

FIGURE 3.4 – Architecture applicative de la plateforme AQI Predictor

L'interface utilisateur est développée avec React et sert à afficher les résultats de prédiction de l'indice de qualité de l'air. Les requêtes utilisateur sont envoyées au backend via des appels HTTP.

Le backend, développé avec FastAPI, traite les requêtes, applique les modèles d'apprentissage automatique et retourne les résultats de prédiction au frontend.

3.5 Pipeline CI/CD

Afin d'automatiser le processus de test, de construction et de déploiement de l'application, un pipeline CI/CD a été mis en place avec GitHub Actions.

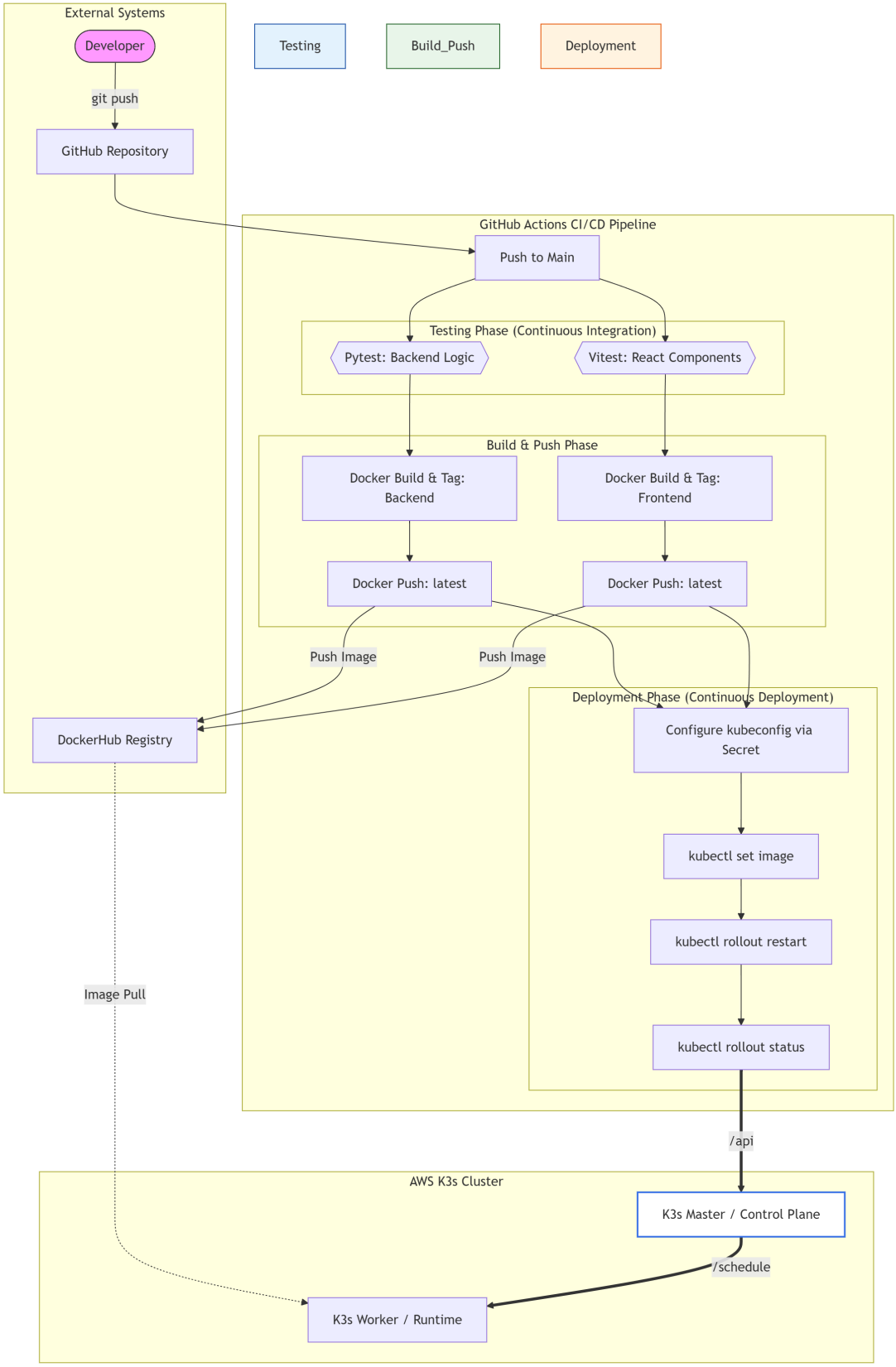

FIGURE 3.5 – Pipeline CI/CD du projet

Le pipeline est déclenché à chaque modification du code source dans le dépôt GitHub. Il exécute automatiquement les tests backend et frontend, construit les images Docker correspondantes, publie ces images dans le registre DockerHub et met à jour les déploiements Kubernetes.

3.6 Déploiement de l'application sur Kubernetes

L'application est déployée dans un namespace Kubernetes nommé `aqi`.

Les manifests Kubernetes définissent :

- Un Deployment pour le backend FastAPI
- Un Service ClusterIP pour exposer l'API backend
- Un Deployment pour le frontend React
- Un Service NodePort permettant l'accès public à l'application

Le frontend agit comme un proxy interne vers l'API backend grâce à une configuration Nginx.

Les sondes de vivacité et de disponibilité permettent à Kubernetes de surveiller l'état du backend via la route `/health`.

Chapitre 4

Mise en place de l'infrastructure et déploiement

Ce chapitre décrit les différentes étapes de mise en place de l'infrastructure DevOps permettant de déployer l'application AQI Predictor sur un cluster Kubernetes léger.

L'infrastructure repose sur une approche *Infrastructure as Code (IaC)* utilisant Terraform pour le provisionnement des ressources cloud et Ansible pour la configuration automatique des serveurs.

L'objectif est de construire un environnement entièrement reproductible permettant de déployer automatiquement l'application conteneurisée sur un cluster Kubernetes basé sur K3s.

4.1 Préparation de l'environnement de développement

Avant de pouvoir déployer l'infrastructure cloud, plusieurs outils doivent être installés sur la machine locale de développement.

L'environnement utilisé dans ce projet repose sur Windows comme système principal avec un environnement Linux via Windows Subsystem for Linux (WSL).

4.1.1 Installation de l'AWS CLI

Avant de pouvoir provisionner l'infrastructure avec Terraform, il est nécessaire d'installer l'outil **AWS CLI** sur la machine locale. Cet outil permet d'interagir avec les services AWS depuis le terminal et constitue une base importante pour la gestion de l'authentification et de la configuration du compte cloud.

Dans ce projet, AWS CLI a été utilisé pour :

- vérifier l'accès au compte AWS
- configurer les identifiants nécessaires au provisionnement de l'infrastructure
- faciliter certaines opérations d'administration et de validation

Sous Windows, l'installation peut être réalisée à partir de l'installeur officiel AWS CLI. Une fois l'installation terminée, la commande suivante permet de vérifier que l'outil est correctement installé :

```
aws --version
```

```
Invite de commandes

Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\sora7>aws --version
aws-cli/2.34.8 Python/3.13.11 Windows/11 exe/AMD64

C:\Users\sora7>
```

Figure 4.1 – Vérification de l'installation de AWS CLI

4.1.2 Configuration des identifiants AWS

Après l'installation de AWS CLI, il est nécessaire de configurer les identifiants d'accès au compte AWS. Cette étape permet aux outils locaux, notamment Terraform, d'authentifier les requêtes envoyées vers l'API AWS.

La configuration est réalisée à l'aide de la commande suivante :

```
aws configure
```

L'utilisateur renseigne alors :

- l'*AWS Access Key ID*
- l'*AWS Secret Access Key*
- la région par défaut, par exemple `us-east-1`
- le format de sortie par défaut

Les informations sont ensuite stockées localement dans les fichiers de configuration de AWS CLI.

Terraform utilise ensuite automatiquement ces identifiants pour communiquer avec AWS lors de l'exécution des commandes `terraform init`, `terraform plan` et `terraform apply`.

Ainsi, AWS CLI joue un rôle de passerelle entre la machine locale de développement et l'infrastructure cloud provisionnée par Terraform.

```
Invite de commandes

Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\sora7>aws configure
AWS Access Key ID [****************65PE]: AKIAQ3VA5MBOO5AI65PE,ik9gRdbKSWEyrDGeYdyc1hu[redacted]
AWS Secret Access Key [****************NiXn]:
Default region name [us-east-1]: us-east-1
Default output format [json]: json

C:\Users\sora7>
```

Figure 4.2 – Configuration des identifiants AWS avec AWS CLI

4.1.3 Création et téléchargement de la clé SSH

Afin de permettre l'administration distante des instances EC2 créées sur AWS, une paire de clés SSH doit être préparée. Cette clé permet d'établir une connexion sécurisée entre la machine locale et les machines virtuelles du cluster.

Dans le cadre du projet, une paire de clés a été associée aux instances EC2 lors de leur création. La clé privée téléchargée au format `.pem` a ensuite été conservée sur la machine locale dans un emplacement sécurisé.

Cette clé SSH est utilisée à deux niveaux :

— pour se connecter manuellement aux instances EC2 à l'aide de la commande `ssh`

— pour permettre à Ansible d'exécuter automatiquement les tâches de configuration sur les nœuds du cluster

Sous Windows, cette clé peut être utilisée directement depuis PowerShell ou depuis WSL. Dans le cas de WSL, il est généralement nécessaire de restreindre les permissions du fichier avant son utilisation :

```
chmod 400 my-key.pem
```

La connexion à une instance EC2 peut ensuite être réalisée avec une commande de la forme suivante :

```
ssh -i my-key.pem ubuntu@<public-dns-or-ip>
```

Dans le rapport, les adresses réelles des machines ne sont pas affichées afin de conserver une présentation générique et orientée automatisation.

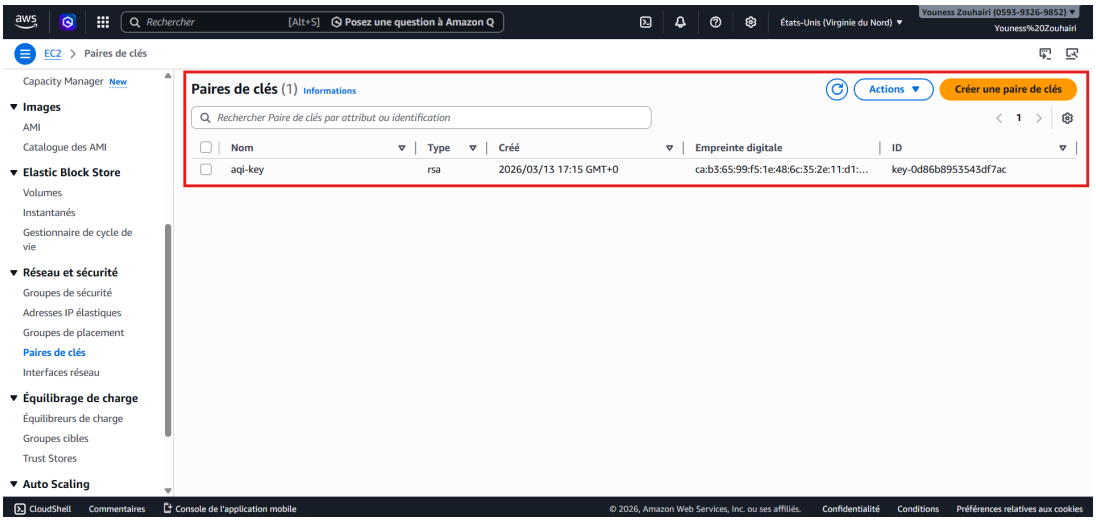

FIGURE 4.3 – Création ou sélection de la paire de clés SSH dans AWS

4.1.4 Installation de Terraform

Terraform est utilisé pour automatiser la création des ressources cloud sur AWS.

Dans ce projet, Terraform a été installé directement sur Windows en téléchargeant le binaire officiel `terraform.exe` depuis le site de HashiCorp.

Le fichier exécutable a ensuite été placé dans un répertoire accessible par le système, puis ajouté à la variable d'environnement `PATH` de Windows afin de pouvoir utiliser Terraform depuis n'importe quel terminal.

Une fois l'installation terminée, la commande suivante permet de vérifier que Terraform est correctement installé :

```
terraform -version
```

Figure 4.4 – Vérification de l'installation de Terraform

4.1.5 Installation d'Ansible sur WSL

Ansible étant principalement conçu pour fonctionner dans un environnement Linux, il a été installé dans une distribution Ubuntu exécutée via Windows Subsystem for Linux (WSL).

L'installation a été réalisée à l'aide du gestionnaire de paquets Ubuntu :

```
sudo apt update
sudo apt install ansible
```

L'utilisation de WSL permet de bénéficier d'un environnement Linux complet tout en travaillant sur un système Windows.

```
youness@BLOB01: ~
youness@BLOB01:~$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/youness/.ansible/plugins/modules', '/usr/share/ansible/plu
gins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/youness/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
youness@BLOB01:~$
```

FIGURE 4.5 – Installation d'Ansible dans WSL

4.1.6 Structure du projet

Le projet est organisé selon une structure modulaire permettant de séparer les différentes responsabilités entre infrastructure, configuration et application.

— **terraform/** : scripts de provisionnement de l'infrastructure AWS

— **ansible/** : configuration automatique des serveurs

— **k8s/** : manifests Kubernetes

— **backend/** : code source de l'API FastAPI

— **frontend/** : application React

— **.github/workflows/** : pipeline CI/CD GitHub Actions

Cette organisation facilite la maintenance et permet une automatisation complète du cycle de déploiement.

4.2 Provisionnement de l'infrastructure avec Terraform

L'infrastructure cloud est provisionnée automatiquement à l'aide de Terraform.

Terraform permet de décrire l'infrastructure souhaitée sous forme déclarative grâce au langage HCL (HashiCorp Configuration Language).

4.2.1 Chaîne d'automatisation entre AWS CLI, Terraform, SSH et Ansible

La mise en place de l'infrastructure repose sur une chaîne d'automatisation cohérente où chaque outil joue un rôle précis.

Le processus peut être résumé comme suit :

1. **AWS CLI** permet de configurer localement les identifiants d'accès au compte AWS.
2. **Terraform** utilise ces identifiants pour provisionner automatiquement les ressources cloud, notamment les instances EC2 et les groupes de sécurité.
3. Lors de la création des instances, une **clé SSH** est associée aux machines afin de permettre leur administration à distance.

4. **Ansible** utilise ensuite cette clé SSH pour se connecter automatiquement aux serveurs créés par Terraform et y installer les dépendances nécessaires.
5. Une fois les nœuds configurés, **K3s** est installé afin de former le cluster Kubernetes.
6. Enfin, les manifests Kubernetes sont appliqués pour déployer l'application conteneurisée.

Cette chaîne d'automatisation garantit une séparation claire entre le provisionnement de l'infrastructure, la configuration des systèmes et le déploiement applicatif, conformément aux bonnes pratiques DevOps.

Ainsi, AWS CLI fournit l'accès au cloud, Terraform crée l'infrastructure, et Ansible exploite cette infrastructure via SSH pour la transformer en environnement d'exécution prêt à accueillir l'application.

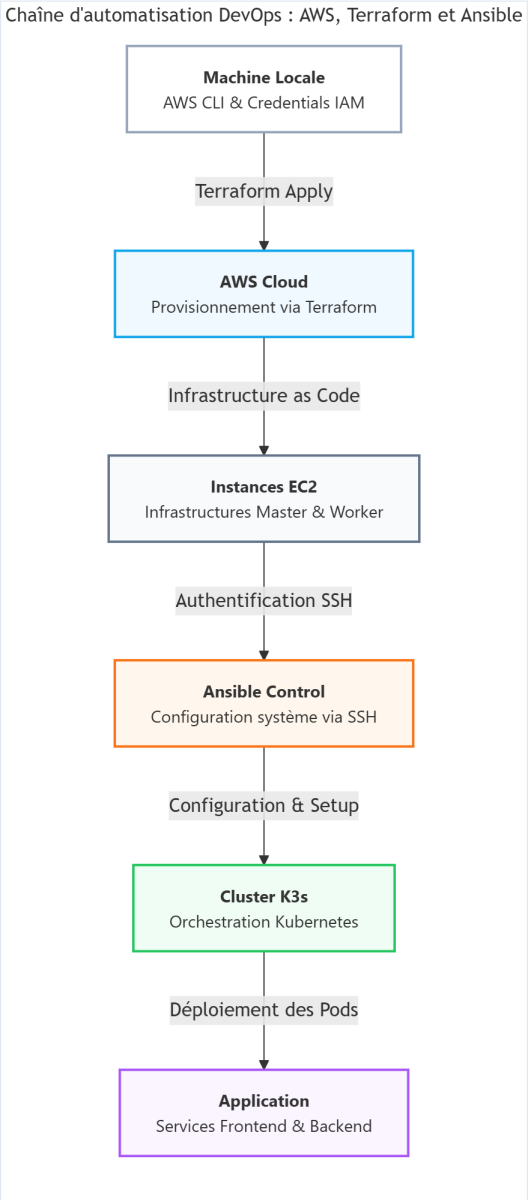

FIGURE 4.6 – Chaîne d'automatisation entre AWS CLI, Terraform, SSH et Ansible

4.2.2 Configuration du provider AWS

Le provider AWS permet à Terraform d'interagir avec l'API AWS afin de créer les ressources nécessaires.

```
provider "aws" {
  region = "us-east-1"
}
```

Terraform s'appuie sur les identifiants AWS configurés localement via AWS CLI pour authentifier les opérations de provisionnement. Ainsi, aucune information sensible n'est directement codée dans les fichiers Terraform, ce qui améliore la sécurité et la portabilité du projet.

4.2.3 Configuration des règles de sécurité

Un groupe de sécurité est configuré afin d'autoriser les communications nécessaires au fonctionnement du cluster Kubernetes et de l'application.

Les ports suivants sont ouverts :

- Port 22 : accès SSH aux machines virtuelles
- Port 6443 : API Server Kubernetes
- Port 80 : accès HTTP à l'application
- Ports 30000–32767 : ports NodePort Kubernetes

Ces règles permettent à la fois l'administration du cluster et l'accès public au frontend de l'application.

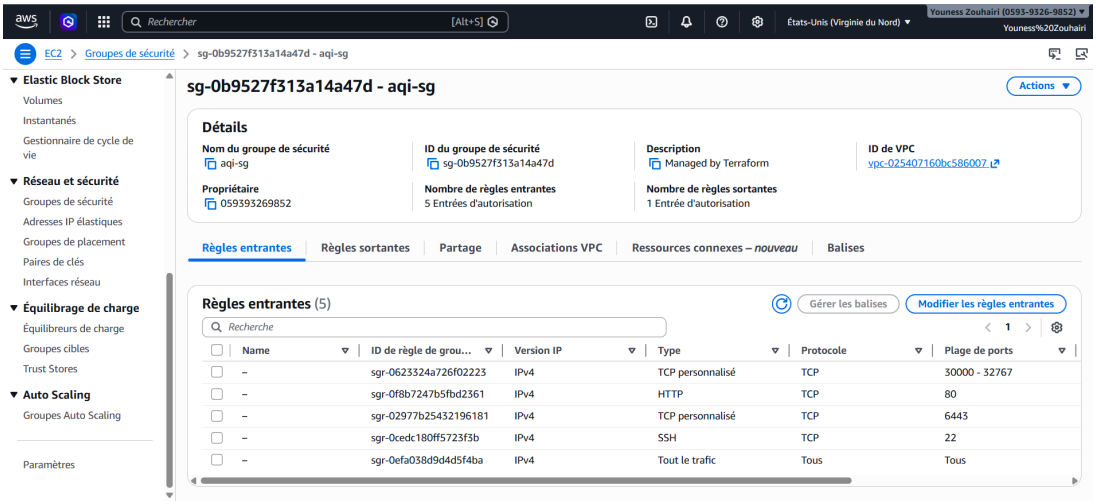

Figure 4.7 – Configuration du Security Group AWS

4.2.4 Création des instances EC2

Deux instances EC2 sont créées afin de constituer le cluster Kubernetes. Lors de leur création, les instances EC2 sont associées à une paire de clés SSH, ce qui permet leur administration distante après le provisionnement. Cette étape est essentielle, car elle constitue le lien entre le provisionnement automatisé réalisé par Terraform et la configuration automatique exécutée ensuite par Ansible.

— **aqi-master** : nœud maître du cluster Kubernetes

— **aqi-worker** : nœud worker hébergeant les pods de l'application

Les deux machines utilisent le type d'instance `t3.micro` inclus dans le programme AWS Free Tier.

Chaque machine dispose d'un volume EBS de **15 Go**.

Ce choix est motivé par la limitation du programme AWS Free Tier qui autorise un maximum de **30 Go de stockage EBS**. En attribuant 15 Go à chaque machine virtuelle, il est possible de créer deux instances tout en restant dans les limites du quota gratuit.

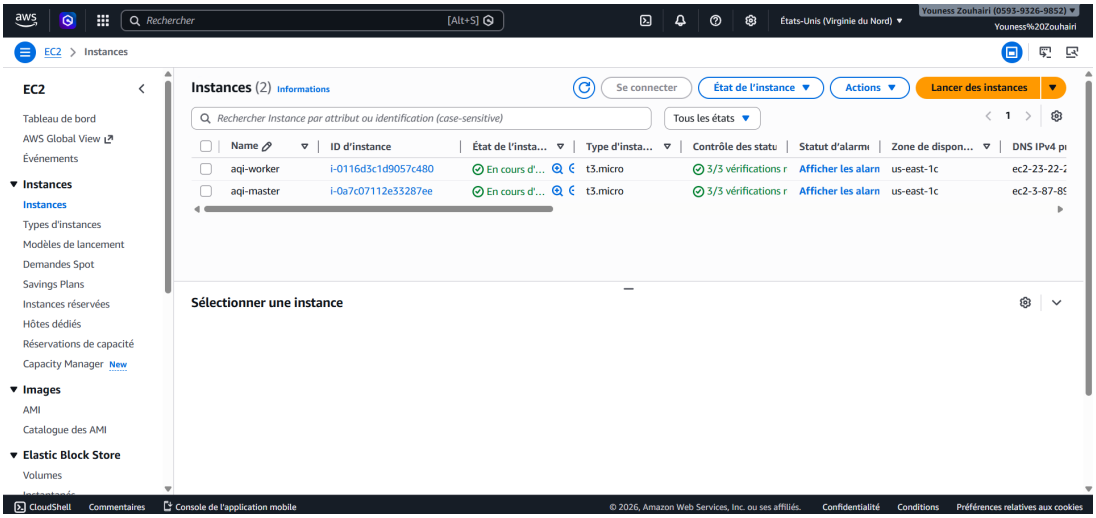

FIGURE 4.8 – Instances EC2 utilisées pour le cluster Kubernetes

4.3 Configuration automatique avec Ansible

Une fois les machines virtuelles créées, Ansible est utilisé pour automatiser leur configuration.

4.3.1 Inventaire Ansible

L'inventaire Ansible permet de définir les machines cibles du playbook.

```
[master]
aqi-master ansible_host=<public-host-master>

[worker]
aqi-worker ansible_host=<public-host-worker>
```

4.3.2 Connexion SSH utilisée par Ansible

Ansible s'appuie sur le protocole SSH pour exécuter à distance les tâches de configuration sur les nœuds du cluster.

Dans ce projet, la clé privée associée aux instances EC2 est utilisée par Ansible pour établir cette connexion sécurisée. Cette approche évite toute configuration manuelle répétitive après le provisionnement des machines.

Un exemple de commande d'exécution du playbook avec spécification de la clé SSH est donné ci-dessous :

```
ansible-playbook -i inventory.ini playbook.yml --private-key my-key.pem
```

Cette étape marque la transition entre le provisionnement de l'infrastructure par Terraform et la configuration logicielle automatisée par Ansible.

Dans une exécution réelle, les adresses publiques ou noms DNS des instances créées par Terraform sont injectés dans l'inventaire Ansible afin de permettre la connexion aux nœuds du cluster. Dans le cadre de ce rapport, ces valeurs sont volontairement généralisées afin de conserver une présentation orientée processus plutôt que détails d'infrastructure spécifiques.

4.3.3 Exécution du playbook

Le playbook Ansible automatise les opérations suivantes :

- création d'un fichier swap
- installation de Docker
- installation de Kubernetes K3s
- initialisation du cluster

— jonction du nœud worker au cluster

— déploiement des manifests Kubernetes

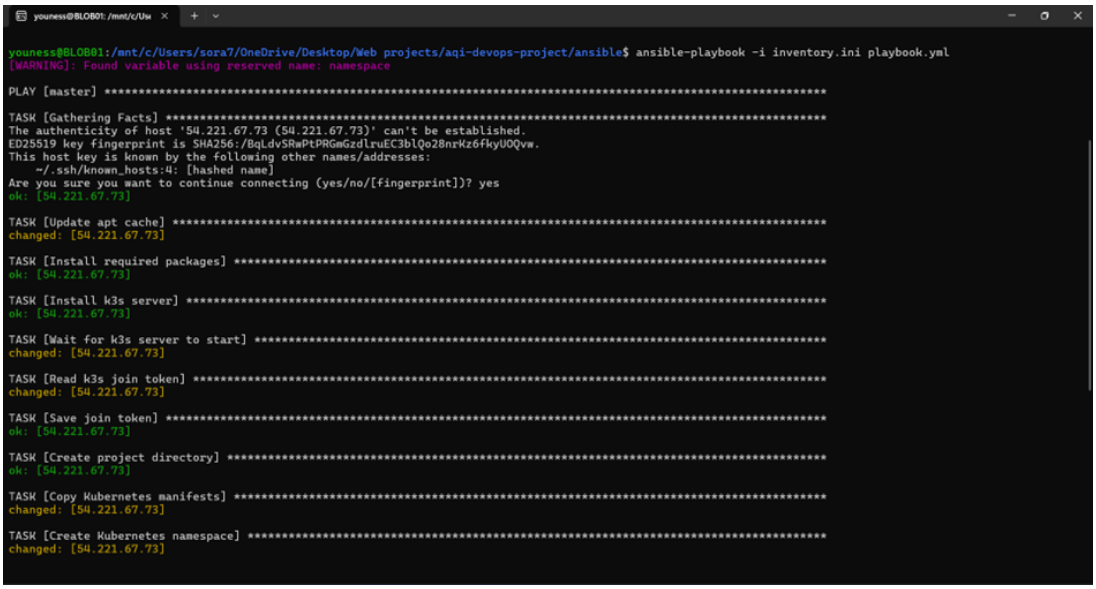

FIGURE 4.9 – Exécution du playbook Ansible

4.3.4 Création d'un fichier swap

Les instances AWS `t3.micro` disposent uniquement de **1 Go de mémoire RAM**. Cette capacité mémoire est insuffisante pour exécuter correctement plusieurs composants Kubernetes.

Afin d'éviter les problèmes liés au manque de mémoire, notamment les erreurs de type *TLS handshake timeout* ou les redémarrages de pods, un fichier swap de **2 Go** est créé sur chaque machine.

Le swap agit comme une extension de la mémoire vive en utilisant le disque lorsque la RAM est saturée.

La création du swap est automatisée via les commandes suivantes :

```
fallocate -l 2G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
```

Cette solution permet d'améliorer la stabilité du cluster malgré les ressources limitées des instances utilisées.

4.4 Déploiement du cluster Kubernetes

Le cluster Kubernetes est installé à l'aide de la distribution légère **K3s**.

K3s est une version simplifiée de Kubernetes conçue pour fonctionner dans des environnements à ressources limitées.

Contrairement à une installation Kubernetes complète, K3s consomme moins de mémoire et moins de CPU, ce qui le rend particulièrement adapté aux instances AWS de petite taille.

4.4.1 Vérification du cluster

Une fois le cluster configuré, la commande suivante permet de vérifier l'état des nœuds :

```
kubectl get nodes
```

Les deux nœuds doivent apparaître avec l'état `Ready`.

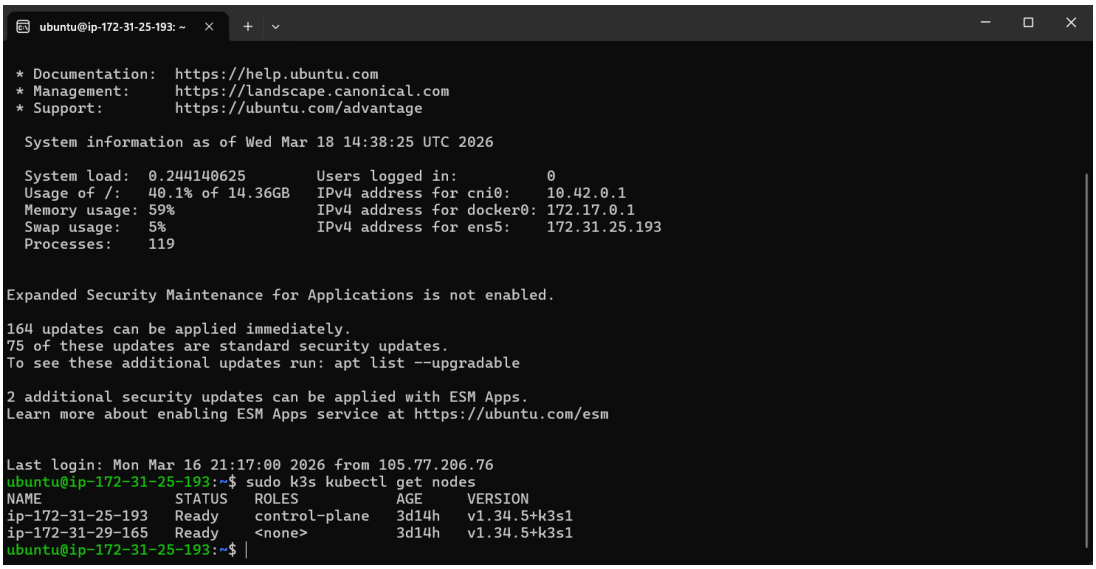

Figure 4.10 – Vérification des nœuds du cluster Kubernetes

4.5 Déploiement de l'application

L'application est déployée dans un namespace Kubernetes nommé `aqi`.

Les manifests Kubernetes définissent les différents composants nécessaires au fonctionnement de l'application.

4.5.1 Deployment du backend

Le backend FastAPI est déployé à l'aide d'un objet Kubernetes `Deployment`. Celui-ci définit l'image Docker utilisée ainsi que les ressources CPU et mémoire allouées au conteneur.

Des sondes de vivacité et de disponibilité sont également configurées afin de vérifier l'état du service via la route `/health`.

4.5.2 Service backend

Le backend est exposé à l'intérieur du cluster via un service Kubernetes de type **ClusterIP**. Cela signifie que l'API est accessible uniquement par les autres services du cluster.

4.5.3 Deployment du frontend

L'interface utilisateur React est servie par un conteneur Nginx et déployée via un Deployment Kubernetes.

4.5.4 Service frontend

Le frontend est exposé à l'extérieur du cluster via un service de type **NodePort**. Ce mécanisme permet d'accéder à l'application en utilisant l'adresse IP publique du nœud worker et le port attribué par Kubernetes.

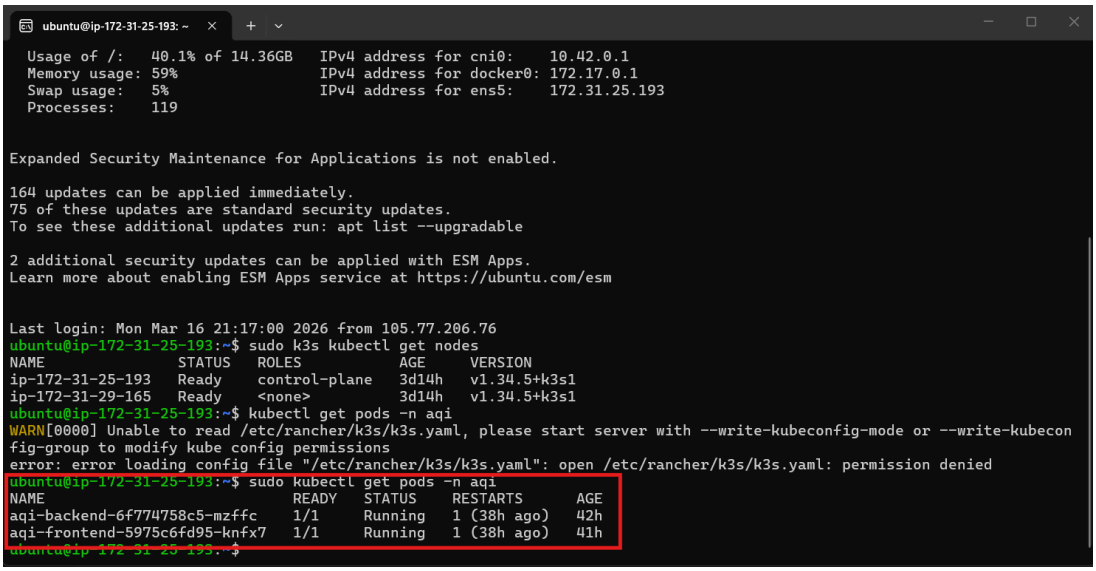

Figure 4.11 – Pods de l'application AQI dans Kubernetes

Chapitre 5

Application conteneurisée et pipeline CI/CD

Ce chapitre présente la conteneurisation de l'application AQI Predictor ainsi que la mise en place d'un pipeline d'intégration et de déploiement continus (CI/CD).

L'objectif est d'automatiser la construction, la publication et le déploiement de l'application afin de réduire les interventions manuelles et de garantir la reproductibilité du système.

5.1 Dépôt GitHub du projet

Le code source du projet est hébergé sur GitHub. Le dépôt contient l'ensemble des composants nécessaires au déploiement de l'infrastructure et de l'application.

Repository : `https://github.com/AshVenn/aqi-devops-project`

Le dépôt contient notamment :

- les scripts Terraform pour la création de l'infrastructure
- les playbooks Ansible pour la configuration des machines
- les manifests Kubernetes
- le backend FastAPI
- le frontend React
- le pipeline CI/CD GitHub Actions

Cette organisation permet de versionner à la fois l'infrastructure et l'application.

5.2 Architecture de l'application

L'application AQI Predictor est composée de deux principaux services :

— un backend développé avec **FastAPI** responsable du calcul et de la prédiction de l'indice de qualité de l'air

— un frontend développé avec **React** permettant l'interaction avec l'utilisateur

Le frontend communique avec le backend via des requêtes HTTP afin d'obtenir les résultats de prédiction.

Chaque service est conteneurisé indépendamment afin de faciliter les mises à jour et le déploiement, comme illustré précédemment.

5.3 Conteneurisation avec Docker

La conteneurisation permet d'encapsuler l'application et toutes ses dépendances dans une image portable.

Cette approche garantit que l'application fonctionne de manière identique sur toutes les machines.

5.3.1 Conteneurisation du backend

Le backend FastAPI est conteneurisé via un Dockerfile définissant l'environnement d'exécution de l'application.

Le Dockerfile réalise les opérations suivantes :

— utilisation d'une image Python légère

— installation des dépendances Python

— copie du code source

— lancement du serveur FastAPI avec Uvicorn

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .
```

```
CMD ["uvicorn","app:app","--host","0.0.0.0","--port","8000"]
```

5.3.2 Conteneurisation du frontend

Le frontend React est construit puis servi via Nginx.

Le processus consiste à compiler l'application puis à servir les fichiers statiques générés.

```
FROM node:18 as builder

WORKDIR /app
COPY package.json .
RUN npm install

COPY . .
RUN npm run build

FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html
```

5.4 Publication des images Docker

Les images Docker sont publiées sur Docker Hub afin d'être accessibles par le cluster Kubernetes.

Deux images principales sont utilisées :

- `aqi-backend`
- `aqi-frontend`

Le cluster Kubernetes télécharge automatiquement ces images lors du déploiement.

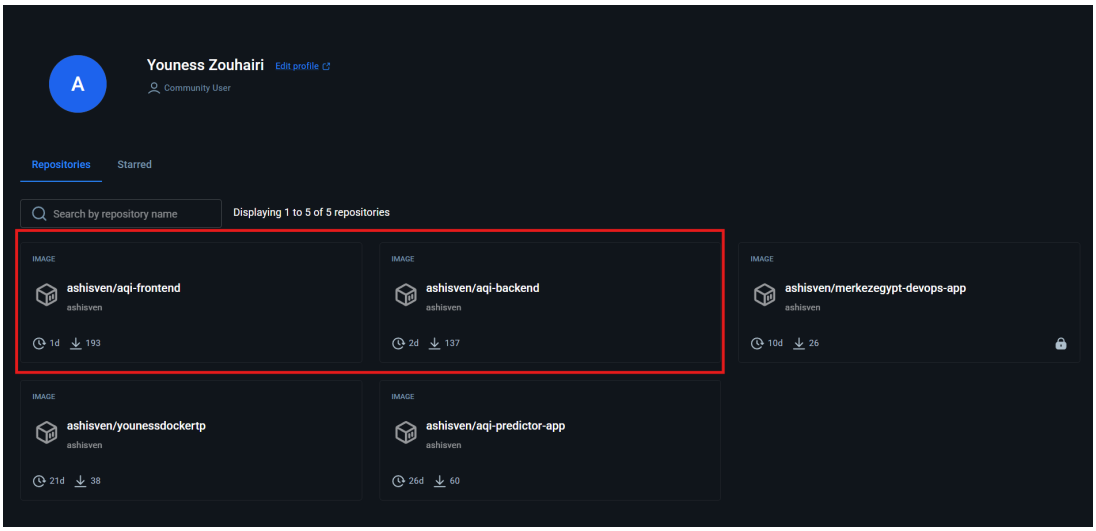

FIGURE 5.1 – Images Docker publiées sur Docker Hub

5.5 Pipeline CI/CD avec GitHub Actions

Afin d'automatiser le processus de déploiement, un pipeline CI/CD a été mis en place à l'aide de GitHub Actions.

Le pipeline est déclenché automatiquement lorsqu'un développeur effectue un `push` sur la branche principale du dépôt.

Le pipeline effectue les opérations suivantes :

1. récupération du code source
2. analyse des fichiers modifiés
3. construction des images Docker nécessaires
4. publication des images sur Docker Hub
5. mise à jour du déploiement Kubernetes

5.6 Authentification de GitHub Actions au cluster Kubernetes

Pour permettre au pipeline GitHub Actions de déployer automatiquement l'application sur le cluster Kubernetes, il a été nécessaire de fournir au pipeline un accès sécurisé au cluster.

Cet accès repose sur le fichier de configuration Kubernetes appelé *kubeconfig*. Ce fichier contient les informations nécessaires pour contacter l'API Kubernetes ainsi que les certificats d'authentification.

Dans le cadre du projet, le fichier *kubeconfig* du nœud maître a été adapté afin d'être utilisable depuis un environnement externe au cluster. Une fois préparé, son contenu a été stocké de manière sécurisée dans les *GitHub Secrets*.

Le pipeline GitHub Actions reconstruit ensuite ce fichier dynamiquement au moment de l'exécution, ce qui permet à la commande `kubectl` de communiquer avec le cluster et de déclencher la mise à jour des déploiements.

Cette approche permet d'automatiser le déploiement sans exposer directement les informations sensibles dans le dépôt GitHub.

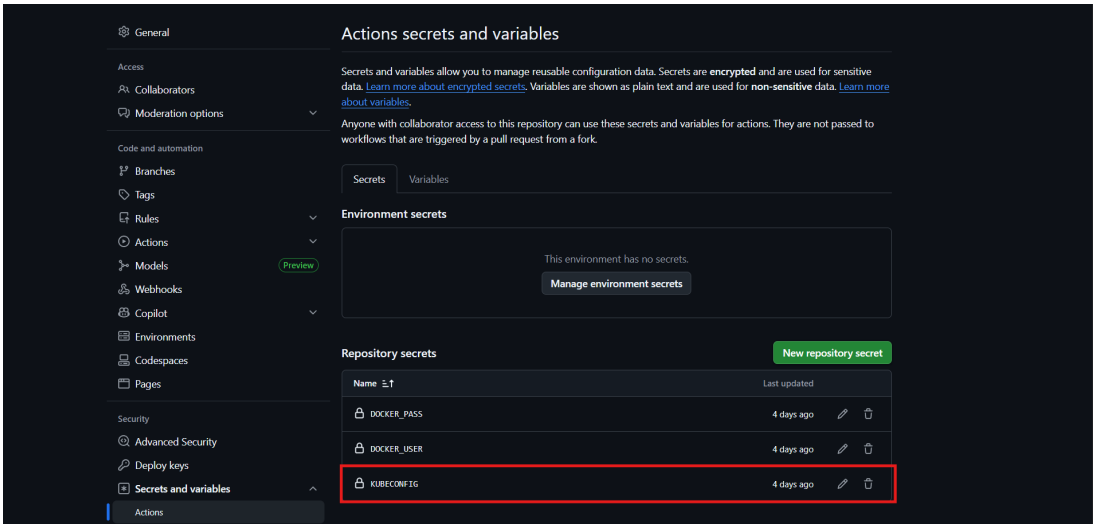

FIGURE 5.2 – Stockage sécurisé du kubeconfig dans GitHub Secrets

5.7 Pipeline conditionnel

Afin d'optimiser les temps de déploiement, le pipeline CI/CD utilise une logique conditionnelle basée sur les fichiers modifiés dans le dépôt.

Trois scénarios sont possibles :

— Si uniquement le dossier `backend/` est modifié, alors le pipeline construit et déploie uniquement l'image backend.

— Si uniquement le dossier `frontend/` est modifié, alors le pipeline construit et déploie uniquement l'image frontend.

— Si les deux dossiers sont modifiés, les deux images sont construites et redéployées.

Cette approche permet de réduire le temps d'exécution du pipeline et d'éviter des reconstructions inutiles.

5.8 Résultat du pipeline GitHub Actions

Lorsqu'un nouveau code est poussé sur le dépôt GitHub, le pipeline se déclenche automatiquement.

Les différentes étapes du pipeline peuvent être observées dans l'interface GitHub Actions.

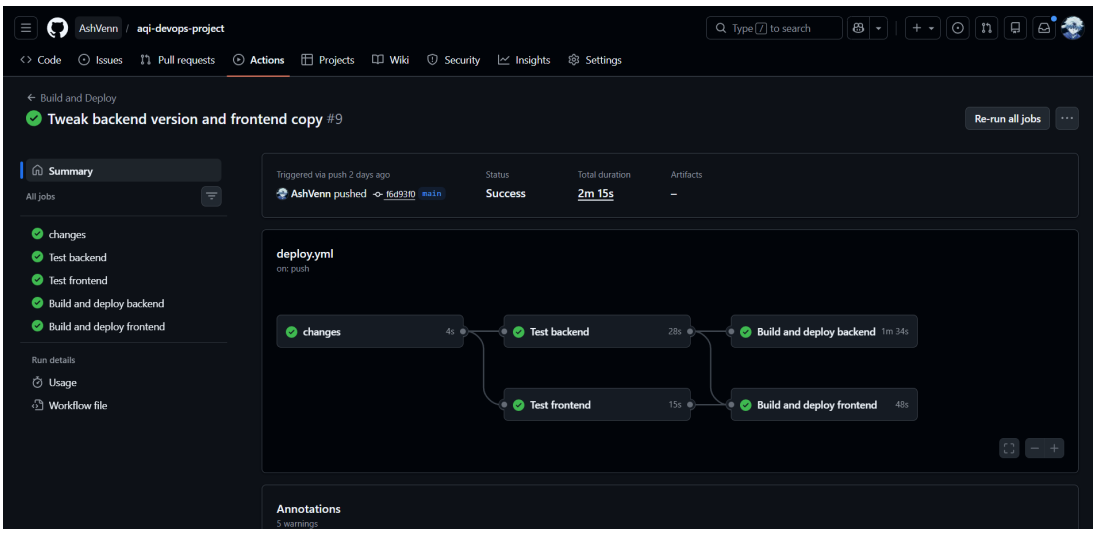

FIGURE 5.3 – Résultat de l'exécution du pipeline CI/CD

Chaque étape du pipeline est exécutée automatiquement :

- construction des images Docker
- authentification auprès de Docker Hub
- publication des images
- déploiement sur Kubernetes

5.9 Déploiement automatique sur Kubernetes

Une fois les images publiées, Kubernetes récupère automatiquement la nouvelle version de l'image et redéploie les pods correspondants.

Cette opération est appelée **Rolling Update**.

Elle permet de mettre à jour l'application sans interruption de service.

```
ubuntu@ip-172-31-25-193: ~

Last login: Wed Mar 18 15:00:24 2026 from 105.77.204.205
ubuntu@ip-172-31-25-193:~$ sudo k3s kubectl logs deployment/aqi-frontend -n aqi
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/18 14:27:57 [notice] 1#1: using the "epoll" event method
2026/03/18 14:27:57 [notice] 1#1: nginx/1.27.5
2026/03/18 14:27:57 [notice] 1#1: built by gcc 14.2.0 (Alpine 14.2.0)
2026/03/18 14:27:57 [notice] 1#1: OS: Linux 6.8.0-1047-aws
2026/03/18 14:27:57 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/18 14:27:57 [notice] 1#1: start worker processes
2026/03/18 14:27:57 [notice] 1#1: start worker process 30
2026/03/18 14:27:57 [notice] 1#1: start worker process 31
ubuntu@ip-172-31-25-193:~$ kubectl get pods -n aqi
WARN[0000] Unable to read /etc/rancher/k3s/k3s.yaml, please start server with --write-kubeconfig-mode or --write-kubecon
fig-group to modify kube config permissions
error: error loading config file "/etc/rancher/k3s/k3s.yaml": open /etc/rancher/k3s/k3s.yaml: permission denied
ubuntu@ip-172-31-25-193:~$ sudo kubectl get pods -n aqi
NAME                              READY   STATUS    RESTARTS      AGE
aqi-backend-6f774758c5-mzffc      1/1     Running   1 (39h ago)   42h
aqi-frontend-5975c6fd95-knfx7     1/1     Running   1 (39h ago)   41h
ubuntu@ip-172-31-25-193:~$
```

Figure 5.4 – Redéploiement automatique des pods Kubernetes

5.10 Limites de l'infrastructure

L'infrastructure utilisée dans ce projet repose sur des instances AWS `t3.micro` appartenant au programme Free Tier.

Ces instances disposent de ressources limitées :

— 1 vCPU

— 1 Go de RAM

Pour cette raison, certaines solutions de monitoring comme Prometheus ou Grafana n'ont pas été déployées car elles nécessitent davantage de ressources.

Afin de limiter la consommation mémoire, la distribution Kubernetes légère **K3s** a été utilisée.

Cette distribution permet d'exécuter un cluster Kubernetes fonctionnel dans un environnement disposant de ressources matérielles limitées.

Chapitre 6

Conclusion générale et perspectives

Ce projet avait pour objectif la conception et le déploiement d'une application intelligente capable d'estimer l'indice de qualité de l'air (AQI) à partir de données environnementales. Au-delà du développement du modèle d'apprentissage automatique, l'accent a été mis sur la mise en place d'une infrastructure DevOps complète permettant de déployer l'application dans un environnement cloud de manière automatisée et reproductible.

L'architecture mise en place repose sur plusieurs technologies modernes telles que Docker, Kubernetes, Terraform, Ansible et GitHub Actions. Ces outils permettent d'automatiser l'ensemble du cycle de vie de l'application, depuis la construction des images Docker jusqu'au déploiement dans le cluster Kubernetes.

Le système final permet ainsi de transformer un simple projet de machine learning en une application web complète, accessible via une interface utilisateur, capable de fournir une estimation de l'indice AQI à partir des données saisies par l'utilisateur.

6.1 Difficultés rencontrées

La réalisation de ce projet a soulevé plusieurs défis techniques liés à la mise en place de l'infrastructure cloud, à la configuration du cluster Kubernetes et à l'automatisation du déploiement.

6.1.1 Contraintes de ressources des instances cloud

Le cluster Kubernetes a été déployé sur des instances AWS de type `t3.micro` appartenant au programme Free Tier. Ces machines disposent de ressources limitées :

— 1 vCPU

— 1 Go de RAM

Cette contrainte a entraîné plusieurs problèmes de performance et de stabilité, notamment des ralentissements, des blocages des sessions SSH et des erreurs de type *TLS handshake timeout* lors de l'accès à l'API Kubernetes.

Solution mise en place :

Afin de stabiliser l'environnement, plusieurs optimisations ont été appliquées :

— utilisation de la distribution Kubernetes légère **K3s**

— création d'un fichier **swap de 2 Go** pour compenser le manque de mémoire

— réduction des ressources allouées aux conteneurs

Ces ajustements ont permis de maintenir un cluster fonctionnel malgré les limitations matérielles.

6.1.2 Accès à l'API Kubernetes

Lors de l'accès au cluster depuis des environnements externes (machine locale ou pipeline CI/CD), des erreurs de type *TLS handshake timeout* empêchaient la communication avec l'API Kubernetes.

Solution mise en place :

Le fichier de configuration Kubernetes (*kubeconfig*) a été ajusté afin de pointer vers une adresse accessible depuis l'extérieur du cluster. De plus, la configuration TLS de K3s a été modifiée afin d'autoriser les connexions externes sécurisées.

6.1.3 Problèmes de communication réseau entre les nœuds

Lors des premiers déploiements, le frontend ne parvenait pas à communiquer avec le backend, ce qui entraînait des requêtes bloquées ou sans réponse.

L'analyse a montré que les règles de sécurité AWS empêchaient la communication interne entre les instances du cluster.

Solution mise en place :

Une règle autorisant le trafic interne entre les machines appartenant au même groupe de sécurité a été ajoutée. Cela a permis de rétablir la communication entre les pods situés sur différents nœuds.

6.1.4 Accès externe à l'application

Dans un premier temps, l'application n'était pas accessible depuis l'extérieur malgré le bon fonctionnement des pods.

Solution mise en place :

Les ports utilisés par les services Kubernetes de type **NodePort** ont été explicitement ouverts dans le Security Group AWS, permettant ainsi l'accès public au frontend.

6.1.5 Crash des conteneurs Nginx

Certains pods frontend entraient dans un état `CrashLoopBackOff` lors du démarrage.

Ce problème était dû au fait que Nginx tentait de résoudre le service backend avant que celui-ci ne soit disponible.

Solution mise en place :

Une configuration Nginx basée sur une variable dynamique a été introduite afin de retarder la résolution du backend jusqu'au moment de la requête, évitant ainsi le crash du conteneur.

6.1.6 Résolution DNS dans Kubernetes

Lors de la configuration du proxy Nginx, des erreurs de résolution de nom de service ont été observées.

Solution mise en place :

Un serveur DNS interne au cluster (CoreDNS) a été explicitement défini dans la configuration Nginx, permettant la résolution correcte des services Kubernetes.

6.1.7 Problèmes de routage des requêtes

Certaines requêtes envoyées par le frontend retournaient des erreurs `404 Not Found`.

Solution mise en place :

Une règle de réécriture des URLs a été ajoutée dans la configuration Nginx afin d'assurer la correspondance entre les routes frontend et backend.

6.1.8 Erreurs de type 502 Bad Gateway

Des erreurs `502 Bad Gateway` ont été observées lors des appels au backend.

Ce problème était lié au temps de réponse du backend, notamment lors du chargement initial du modèle de machine learning.

Solution mise en place :

Le temps d'attente du proxy Nginx a été augmenté afin de permettre au backend de répondre correctement, même en cas de latence due au chargement du modèle.

6.1.9 Synchronisation des services Kubernetes

Dans certains cas, les services Kubernetes ne détectaient pas les pods, ce qui empêchait le routage des requêtes.

Solution mise en place :

Une vérification systématique des labels a été réalisée afin de garantir la cohérence entre les *Deployments* et les *Services*.

```
kubectl get pods --show-labels
```

6.1.10 Optimisation du pipeline CI/CD

La mise en place du pipeline CI/CD a soulevé un problème d'efficacité lié à la reconstruction systématique des images Docker.

Solution mise en place :

Une logique conditionnelle a été intégrée afin de détecter les fichiers modifiés :

- reconstruction uniquement du backend si `backend/` est modifié
- reconstruction uniquement du frontend si `frontend/` est modifié
- reconstruction des deux services si nécessaire

Cette optimisation permet de réduire le temps d'exécution du pipeline et d'améliorer l'efficacité du déploiement.

6.2 Apports du projet

Ce projet a permis d'acquérir une expérience concrète dans plusieurs domaines clés du DevOps et du cloud computing :

- mise en place d'une infrastructure cloud automatisée
- déploiement d'un cluster Kubernetes
- conteneurisation d'applications avec Docker
- automatisation du déploiement avec GitHub Actions
- gestion de l'infrastructure avec Terraform et Ansible

Il démontre également comment un projet de machine learning peut être transformé en une application web complète déployée dans un environnement cloud.

6.3 Perspectives d'amélioration

Bien que l'application développée soit fonctionnelle, plusieurs améliorations peuvent être envisagées pour des déploiements futurs.

6.3.1 Ajout d'un système de monitoring

Dans un environnement de production réel, il serait souhaitable d'ajouter une solution de monitoring basée sur :

- Prometheus pour la collecte des métriques
- Grafana pour la visualisation
- Alertmanager pour la gestion des alertes

Ces outils permettraient de superviser l'état du cluster Kubernetes et d'anticiper les problèmes de performance.

6.3.2 Mise en place d'un Ingress Controller

Actuellement, l'application est exposée via un service Kubernetes de type NodePort. Une amélioration consisterait à déployer un **Ingress Controller** permettant :

— l'utilisation d'un nom de domaine

— la gestion du trafic HTTP

— l'intégration de certificats HTTPS

6.3.3 Extension du modèle de prédiction

Le modèle actuel fournit une estimation de l'indice AQI à l'instant présent. Une amélioration possible serait de développer un modèle de **prévision temporelle** permettant d'anticiper l'évolution de la qualité de l'air dans les heures ou les jours à venir.

Cela nécessiterait notamment l'utilisation de modèles de séries temporelles et la construction de variables retardées (*lags*).

6.4 Conclusion

Ce projet démontre la faisabilité de la mise en place d'une plateforme complète combinant intelligence artificielle, cloud computing et pratiques DevOps modernes.

La combinaison de Docker, Kubernetes et GitHub Actions permet d'automatiser efficacement le déploiement d'une application de machine learning dans un environnement cloud.

Malgré les contraintes matérielles liées à l'utilisation d'instances Free Tier, les solutions mises en place ont permis de construire une architecture stable et fonctionnelle.

Cette expérience constitue une base solide pour le développement et le déploiement de systèmes intelligents à plus grande échelle.